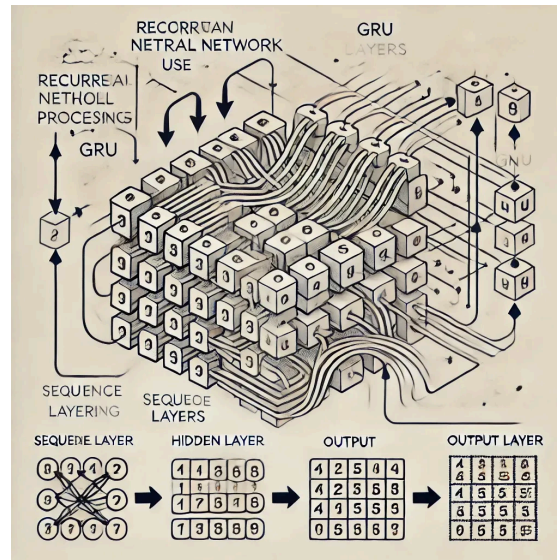


Building a Recurrent Neural Network (RNN) with PyTorch

Abhinav Mishra · Follow
3 min read · Sep 12, 2024

2

Recurrent Neural Networks (RNNs) are widely used for sequence data tasks such as time series forecasting, language modeling, and more. In this post, I will walk you through how to build and train an RNN using PyTorch based on the MNIST dataset, which consists of handwritten digit images.



Imports

First, let's import the necessary libraries:

```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import DataLoader
import torchvision.datasets as datasets
import torchvision.transforms as transforms
```

These imports include PyTorch's core libraries, optimization functions, and the MNIST dataset from torchvision.

Creating the RNN Model

We'll define a class `RNN`, which inherits from `nn.Module`. The model will use a GRU (Gated Recurrent Unit), a type of RNN, to process sequence data. We also include a fully connected layer to output the final classification.

```
# Create a RNN
class RNN(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers, num_classes):
        super(RNN, self).__init__()
        self.hidden_size = hidden_size
        self.num_layers = num_layers
        self.gru = nn.GRU(input_size, hidden_size, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size * sequence_length, num_classes)

    def forward(self, x):
        h0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size).to(device)

        # Forward Prop
        out, _ = self.gru(x, h0)
        out = out.reshape(out.shape[0], -1)
        out = self.fc(out)

        return out
```

- **GRU:** We use a GRU layer for handling the sequential nature of the data.
- **Forward Function:** This is where the data flows through the network. The hidden state is initialized, passed into the GRU, and then flattened for the fully connected layer.

Setting the Device

We set up the device to run on a GPU if available:

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

Model Hyperparameters

Next, we define some key parameters:

```
input_size = 28
sequence_length = 28
num_layers = 2
hidden_size = 256
num_classes = 10
learning_rate = 0.001
batch_size = 64
num_epochs = 2
```

- **input_size** and **sequence_length**: MNIST images are 28x28, and we treat the rows as a sequence of 28 inputs.
- **hidden_size**: The number of units in the GRU's hidden layer.
- **num_layers**: Number of GRU layers.
- **num_classes**: For MNIST, there are 10 digit classes (0–9).
- **learning_rate**, **batch_size**, and **num_epochs** are common training hyperparameters.

Loading the Dataset

We use the MNIST dataset for training and testing. PyTorch's `DataLoader` helps load and batch the data.

```
train_dataset = datasets.MNIST(root='dataset/', train=True,
                                transform=transforms.ToTensor(), download=True)
train_loader = DataLoader(dataset=train_dataset, batch_size=batch_size,
                           shuffle=True)

test_dataset = datasets.MNIST(root='dataset/', train=False,
                                transform=transforms.ToTensor(), download=True)
test_loader = DataLoader(dataset=test_dataset, batch_size=batch_size,
                          shuffle=True)
```

Initializing the Model

With the `RNN` class defined, we instantiate the model:

```
model = RNN(input_size, hidden_size, num_layers, num_classes).to(device)
```

Loss Function and Optimizer

For this classification task, we use cross-entropy loss and the Adam optimizer:

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)
```

Training the RNN

We now train the model. For each batch, we feed the data into the model, compute the loss, and update the weights through backpropagation.

```
for epoch in range(num_epochs):
    for batch_idx, (data, targets) in enumerate(train_loader):
        # Get data to cuda if possible
        data = data.to(device=device).squeeze(1)
        targets = targets.to(device=device)

        # forward
        scores = model(data)
        loss = criterion(scores, targets)

        # backward
        optimizer.zero_grad()
        loss.backward()

        # gradient descent or adam step
        optimizer.step()
```

- **squeeze(1)**: This removes the unnecessary channel dimension from the MNIST images.
- **optimizer.step()**: This applies the gradient descent step to update the weights.

Checking Accuracy

After training, we check the model's accuracy on both the training and test data:

Open in app ↗

```
# Function to check accuracy
def check_accuracy(loader, model):
    if loader == train_loader:
        print("Checking accuracy on training data")
    else:
        print("Checking accuracy on test data")

    num_correct = 0
    num_samples = 0
    model.eval()

    with torch.no_grad():
        for x, y in loader:
            x = x.to(device).squeeze(1) # Reshape the input tensor
            y = y.to(device)

            # Forward pass
            scores = model(x)
            _, predictions = scores.max(1)
            num_correct += (predictions == y).sum()
            num_samples += predictions.size(0)

    accuracy = float(num_correct) / float(num_samples) * 100
    print(f'Got {num_correct} / {num_samples} correct with accuracy {accuracy:.2f}%')
    model.train()

# Check accuracy
check_accuracy(train_loader, model)
check_accuracy(test_loader, model)
```

This function evaluates the model on the given data loader. It switches the model to evaluation mode and calculates the accuracy by comparing predictions with actual labels.

Results

After training for just 2 epochs, the model achieves impressive results:

Checking accuracy on training data
Got 59060 / 60000 correct with accuracy 98.43%
Checking accuracy on test data
Got 9827 / 10000 correct with accuracy 98.27%

This shows that even a simple RNN with GRU layers can achieve high accuracy on the MNIST dataset.

Conclusion

In this post, we've built an RNN using PyTorch to classify images from the MNIST dataset. By using a GRU, we can effectively handle sequence data, and the model performs very well, achieving over 98% accuracy on the test set. With more training or model tuning, this accuracy could potentially be improved even further.

- Deep Learning
- Neural Networks
- Rnn
- Pytorch
- Machine Learning



Written by Abhinav Mishra

3 Followers · 2 Following

PyTorch | Machine Learning | GenAI

Follow

No responses yet



Sandipan Dey

What are your thoughts?

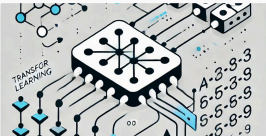
More from Abhinav Mishra



Bidirectional LSTM for MNIST Classification using PyTorch

In this post, we'll dive into how to implement a Bidirectional LSTM (Long Short-Term...

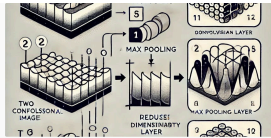
Sep 14, 2024



Understanding Transfer Learning: Leveraging Pre-trained Models fo...

Transfer learning is a powerful technique in deep learning that allows us to reuse pre...

Sep 13, 2024



Abhinav Mishra

Understanding a Simple CNN using PyTorch: Step-by-Step Guide

In this blog post, I will walk you through how I built and trained a simple Convolutional...

Sep 12, 2024 10



Abhinav Mishra

PyTorch vs TensorFlow: A Clear Comparison for Deep Learning...

As a machine learning practitioner, you've probably come across both PyTorch and...

Sep 25, 2024



See all from Abhinav Mishra

Recommended from Medium



In Artificial Intelligence in Plain E... by Nidhi Gahla...

Everything you need to know about Logistic Regression!

Understanding the Fundamentals of Classification Algorithms and Their Role in...

Oct 8, 2024 51



noplaxochia

Numpy multilayer perceptron from scratch

With backpropagation

Oct 14, 2024

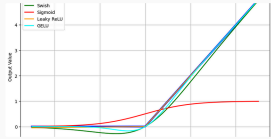


In AI Advances by Amit Kulkarni

Revolutionize Your Spreadsheets: Get Instant Insights with Quadrati...

Ditch Static Spreadsheets! See How Quadratic AI Uses AI & Live Data to...

4d ago 187

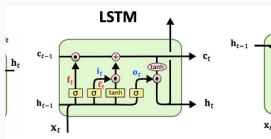


Hugman Sangkeun Jung

Understanding Activation Functions

Mastering Activation Functions: From Basics to Advanced Neural Network Concepts

Nov 15, 2024



LM Po

A Journey Through RNN, LSTM, GRU, and Beyond

If you're not a Medium subscriber, click here to read the full article.

Feb 14 19



Cristian Leo

The Math Behind Transformers

Deep Dive into the Transformer Architecture, the key element of LLMs. Let's explore its...

Jul 25, 2024

1K 9



See more recommendations